

Data Analytics Technology Indexing and Tuning

Catarina Silva

catarina@dei.uc.pt

2026

DW tuning objects and techniques

Special indexes: bitmaps,
bitmap join indexes

Index-Organized Tables

Cluster Tables: avoiding joins

Materialized views

DW tuning objects and techniques

Parallel processing

Active and archived data separation

Duplication of columns

Pre-computed summary columns

Vertical partitioning

Indexing

B-tree Indexes

Index Cost Model

Bitmap Indexes

Join Indexes

Function-Based Indexes

Index-Organized Tables

Clustered Tables

Bulk loading and indexing

Indexing - Definition

- Tree structure that allows faster direct access to at least one column of a table
- Speed up data access
- Can ensure data uniqueness
- Independent (both logically and physically) of the data in the table to which they are associated
- Automatically updated whenever data is inserted, updated and/or deleted from the table to which they are associated
- They impair performance in DML (Data Manipulation Language) commands

Indexing – Implementation (1)

- One of the best techniques for achieving good database performance
- This involves creating the appropriate indexes for the database
 - How many indexes should be created?
 - How should each index be defined?
- Method to be used
 - for large tables
 - when queries are very specific
 - when queries that access $\leq 25\%$ of the table's rows
 - for essentially static tables

Indexing – Implementation (2)

- A table can have multiple indexes
- Which columns should be indexed?
 - frequently accessed, especially with WHERE clauses (SQL)
 - containing many null values, but the non-null values are accessed by most queries
 - FOREIGN KEYS
 - PRIMARY KEYS
 - CANDIDATE KEYS
 - JOINS, ORDER BY, GROUP BY, UNION and DISTINCT
 - where most values are distinct
 - with a wide range of values
 - LONG and LONG RAW columns are not indexable!

Indexing – Implementation (3)

- Minimise the number of indexes (queries versus DML operations)
- Column indexing order
 - Place the most frequently used columns first
- Example
 - Several queries use PRODUCT_TYPE
 - Few queries use PRODUCT_TYPE and PRODUCT_COLOUR
 - Create an index with the combination (PRODUCT_TYPE, PRODUCT_COLOUR)

Indexing - Cost

- Overhead in index updates whenever rows are inserted, updated or deleted from the table
- Additional space required for index storage
 - Worst-case scenario: indexes taking up more space than the table data
- Data loading and table reorganisation operations will take much longer
- The database will require additional files for storage
 - May cause problems in the OS (too many open files)

Indexing Classification

Logical

- Simple or composite
- Unique or duplicate

Physical

- Partitioned or non-partitioned
- B-tree [reverse key] or bitmap
- Function-based
 - include columns transformed by a function
 - include an expression (e.g. col1+col2)

B-tree indexes

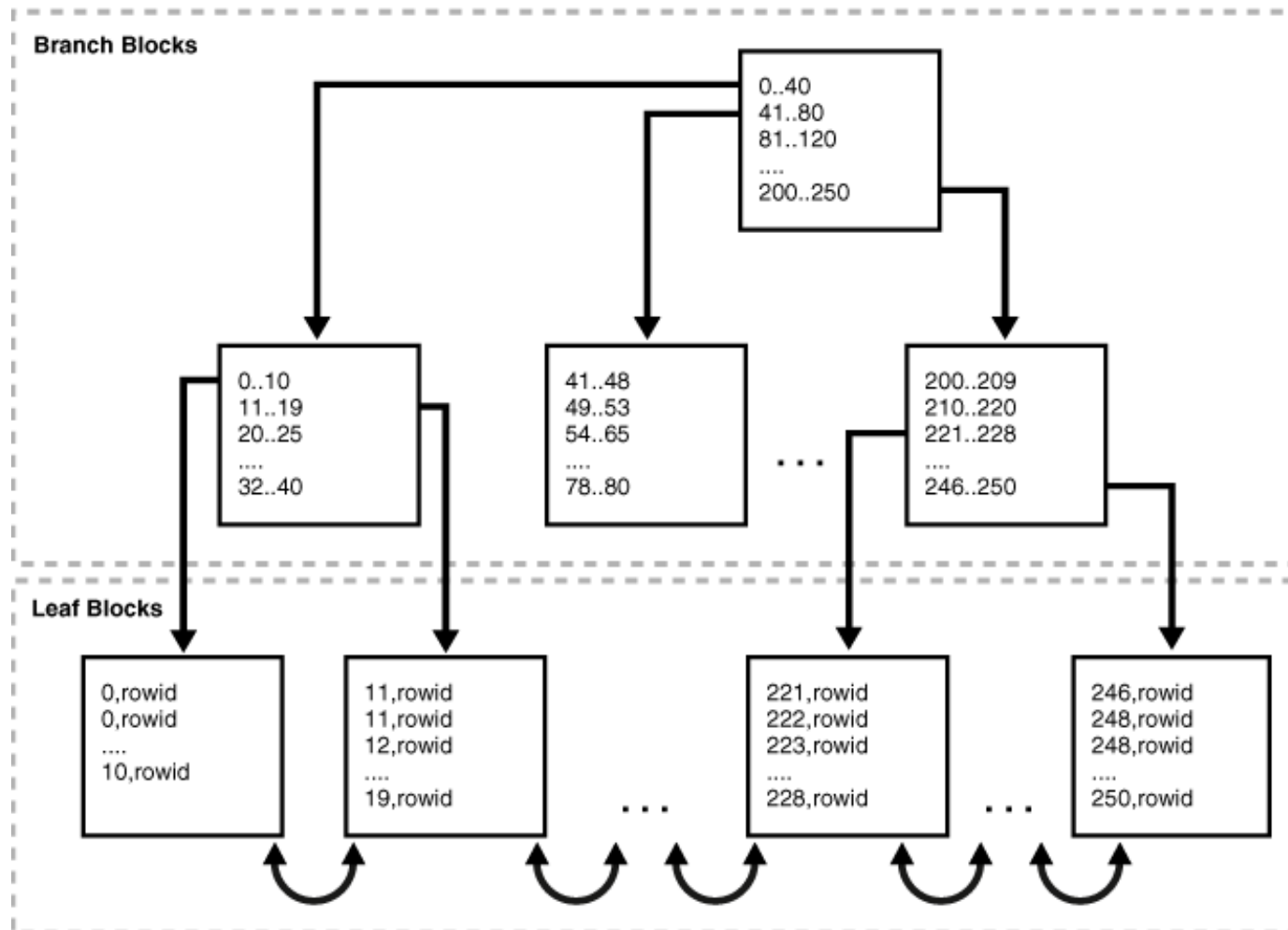
Structure

- root
- branches
- leaves

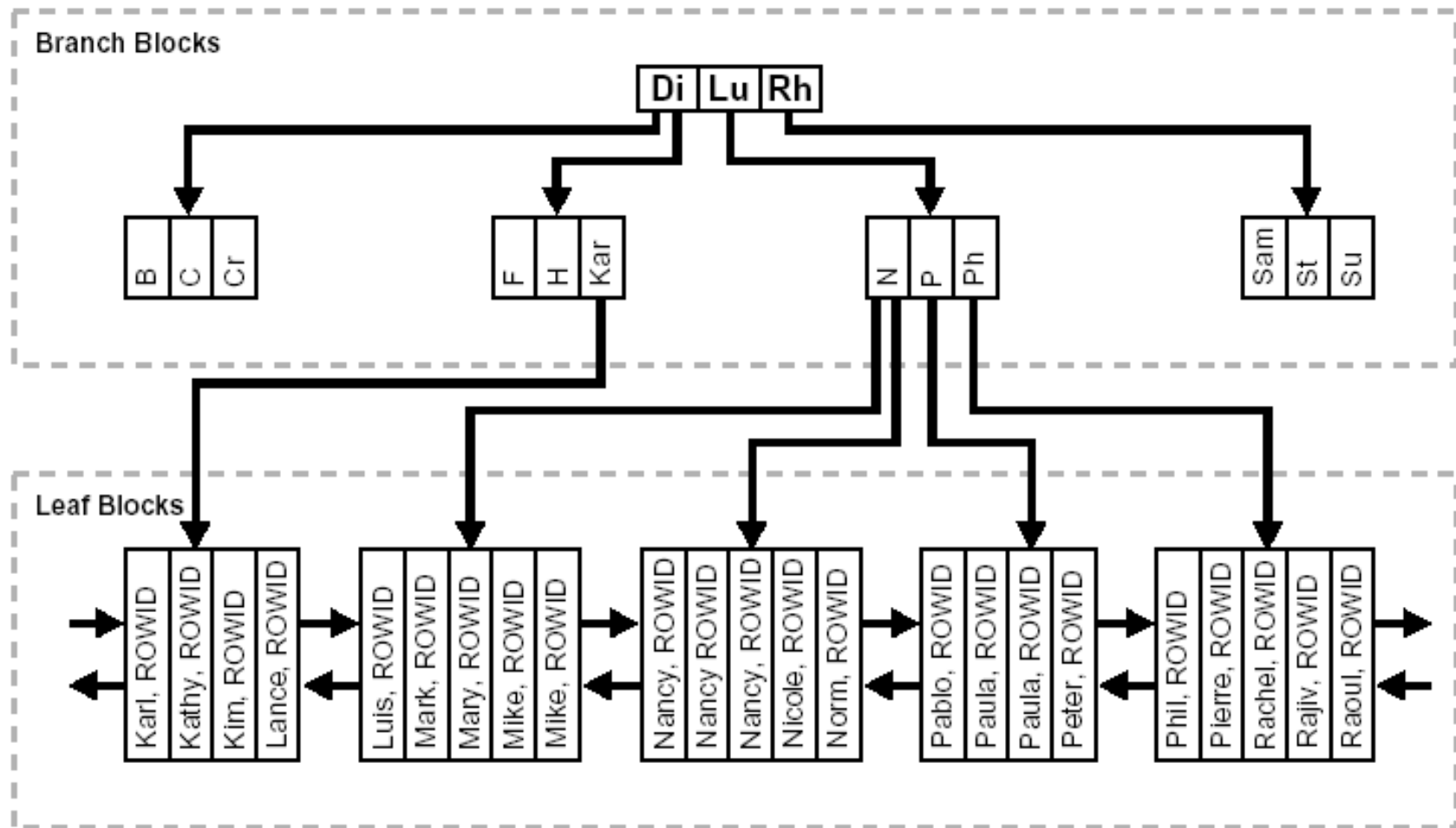
Format of index entries (leaves)

- header
 - Status (locked or unlocked)
 - number of columns
- Data (single-column indexes)
 - column size (bytes)
 - column value
- ROWID of the rows containing a specific value

B-tree index example



B-tree index example



B*-Tree indexes

Indexes are automatically created for:

- The table primary key
- Columns with the UNIQUE constraint

An index can be UNIQUE or NON-UNIQUE:

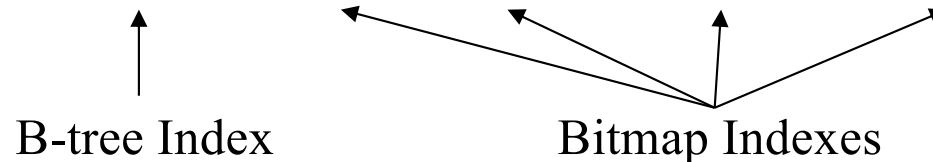
- UNIQUE indexes guarantee that the values in indexed columns are unique
- NON-UNIQUE indexes allows repeated values in the indexed columns

Bitmap indexes

- Used when the cardinality of the attributes is very low
 - Cardinality = number of distinct values

Client

<i>ID_Client</i>	<i>Gender</i>	<i>District</i>	<i>MS</i>	<i>Income</i>
145023	M	Coimbra	Married	A
145025	M	Faro	Single	B
154265	F	Guarda	Married	A
265453	M	Faro	Single	C
645654	F	Beja	Single	A



Bitmap indexes

- Structure
 - Organised like a B-tree
- Index entry format
 - header
 - Status (locked or unlocked)
 - number of columns
 - Data
 - column size
 - column value
 - start ROWID, end ROWID
 - segment bitmap
 - bit string

Bitmap indexes examples

- Indexes for attributes Gender and Income Category
- Attributes like District and Marriage_Status are also candidates for Bitmap indexing

Gender='M'	Gender='F'	Income='A'	Income='B'	Income='C'
1	0	1	0	0
1	0	0	1	0
0	1	1	0	0
1	0	0	0	1
0	1	1	0	0

Gender = 'M'		Income = 'A'		Income = 'B'
1		1		0
1		0		1
0	AND	1	OR	0
1		0		0
0		1		0

```

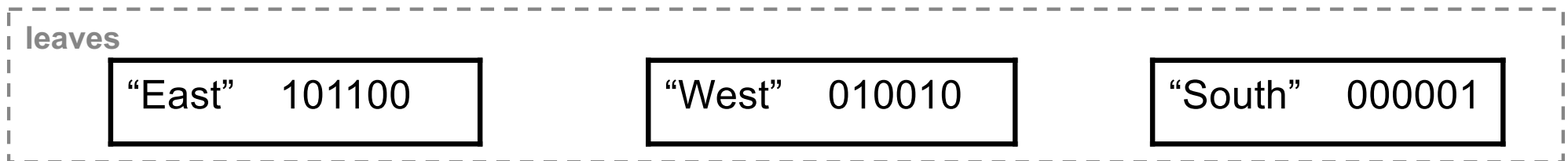
SELECT COUNT(*)
FROM Client
WHERE Gender = 'M' AND income IN ('A','B');

```

Bitmap index example

#Client	Marital status	Region	Sex
101	Single	East	M
102	Single	West	F
103	Married	East	M
104	Single	East	F
105	Divorced	West	F
106	Married	South	M

Bitmap index for “Region”



Bitmap vs B-tree

- Bitmaps only allow equality matches
- Bitmaps much smaller
 - ... Bitmap proportional to $|\text{values}| * |\text{rows}|$
 - ... B-tree proportional to $|\text{rows}|$
- Bitmaps can be efficiently combined

B-TREE	BITMAP
High-cardinality columns	Low-cardinality columns
Updating keys is inexpensive	Updating keys is very expensive
Inefficient for OR-operator queries	Efficient for OR-operator queries
Used in OLTP environments (dynamic tables)	Used in DSS environments (large-volume static tables)

Index creation (b-tree)

- Implicit
 - PRIMARY KEY and UNIQUE KEY
- Explicit (SQL):

```
CREATE [UNIQUE] INDEX [schema.]name_index
ON [schema.] table_name
    (column [ASC | DESC] [, ...] )
[TABLESPACE name_tab]
[STORAGE storage_clause]
[LOGGING | NOLOGGING] [NOSORT] ;
```

Index creation (bitmap)

```
CREATE BITMAP INDEX [schema.]name_index  
ON [schema.]name_table  
    (column [ASC | DESC] [, ...] )  
[TABLESPACE name_tab]  
[STORAGE storage_clause]  
[LOGGING | NOLOGGING]  
[NOSORT] ;
```

Examples – index creation

```
CREATE INDEX IdxTCliente_Apelido ON TCliente  
(Apelido) TABLESPACE indx01;
```

```
CREATE UNIQUE INDEX IdxTCliente_Email  
ON TCliente (Email) TABLESPACE indx03;
```

```
CREATE BITMAP INDEX IdxTConselho_NomeConcelho  
ON TConcelho (NomeConcelho) TABLESPACE indx02;
```

Examples – index creation

```
CREATE INDEX IdxUpperNome ON TCliente  
(UPPER(Apelido));
```

```
SELECT * FROM TCliente  
WHERE UPPER(Apelido) = 'SILVA';
```

```
CREATE INDEX IdxExemplo ON Tab_X  
(colA + colB * (colC - 1), colA, ColB );
```

```
SELECT colA FROM Tab_X  
WHERE colA + colB * (colC - 1) < 100;
```

Index reconstruction

- Move the index to a different tablespace
- Free up memory space from deleted entries

```
ALTER INDEX [schema.]name_index REBUILD  
    [TABLESPACE name_tab]  
    [STORAGE storage_clause]  
    [LOGGING | NOLOGGING];
```

```
ALTER INDEX IdxTCliente_Apelido REBUILD  
    TABLESPACE indx04;
```

Indexes over several columns

- An index can be created over more than one column of a given table
- Example:

```
create index idx_prod_id  
on prod_stores (idprod, idstore);
```

- Useful when there are columns that are frequently accessed simultaneously
 - Specially if those columns are used in conditions

Order of columns in indexes

- prod_stores (IDProd, IDStore, price, quantity)
 - The table has a few tens of stores (IDStore)
 - Each store has thousands of products (IDProd)
- Frequent query:

```
select * from prod_stores
where idprod = 1243
and idstore = 8;
```

- The index should use the column with more distinct values (IDProd) first

```
create index idx_prod_stores
on prod_stores (idprod, idstore);
```

B-tree cost model

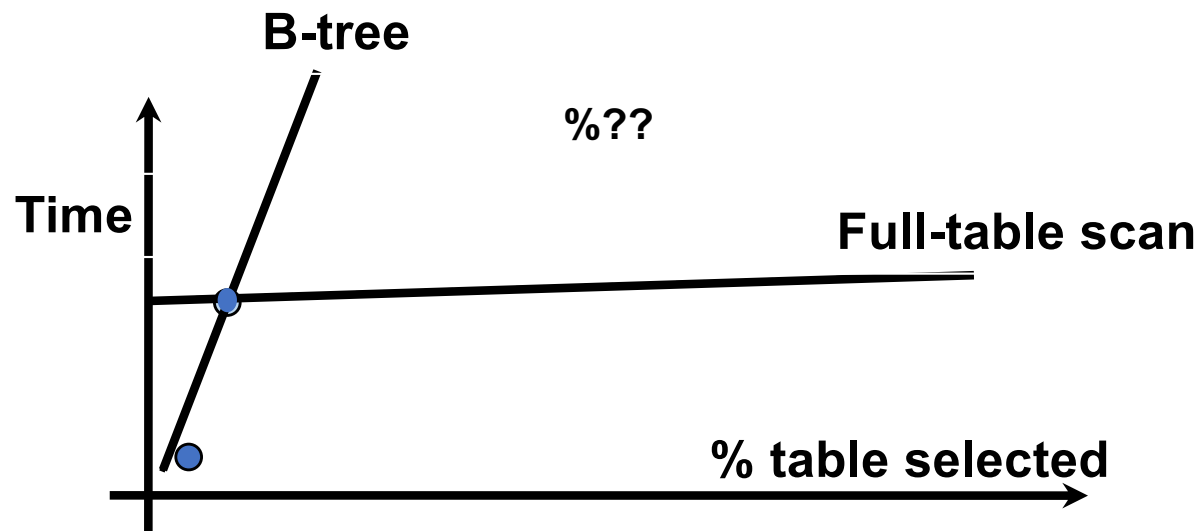
- Disk-optimized structure
- Nodes as big as (disk) pages
- Fan-out: number of pointers per node
 - The higher the better → less levels, less IO
- Typically:
 - 3-4 levels
 - Root in memory
- Random IO (10-20 ms)
- Sequential IOs (~0-2 ms)

Operations

- Find in index: 2-3 random IOs
- Table lookup: find+1 random IO
- Range search: lookup + n sequential IOs
- Insert: find + potencial cascade splits
- Delete: find + potencial cascade merges

Index vs full-table scan

- Seek times versus transfer rates
 - Fundamental decision: b-tree or full-table scan?
 - Cost of b-tree: 3-4 random IOs
 - Cost of full-table-scan: N sequential IOs



Which tables to index?

Obvious: Big tables (with many rows)

Tables for which the queries return a small percentage of the records

Tables with few data manipulation operations and lots of queries

Indexes slow down data manipulation operations

Which columns should be indexed?

Columns frequently used in queries

Primary keys

Foreign keys

Columns with many different values

Columns with many null values in which queries normally access the rows with non-null values

When and how many indexes?

- Indexes can be created at any time
- The best option is to create the indexes after loading the table for the first time
- The number of indexes in a table is unlimited
- Indexes improve performance, but not always!
- Q: Should index every column?...

Dropping indexes

```
drop index index_name;
```

- Dropping an index does not change the data
 - Only affects the performance of the operations
- When one table is dropped all the associated indexes are dropped

Dropping indexes

- An index should be dropped when:
 - It did not improve the performance as expected
 - May turn off index instead
 - Applications do not perform anymore the queries that needed the index
 - The table is going to suffer intensive inserts/updates/deletes. the index should be rebuilt latter on
 - Is fragmented: it should be dropped and recreated

Challenge 1 – index definition

What indexes should be used to optimize data access and why. Note that it is not enough to indicate the type of structures to use, it is necessary to concretize saying on which tables and fields these structures should be created.

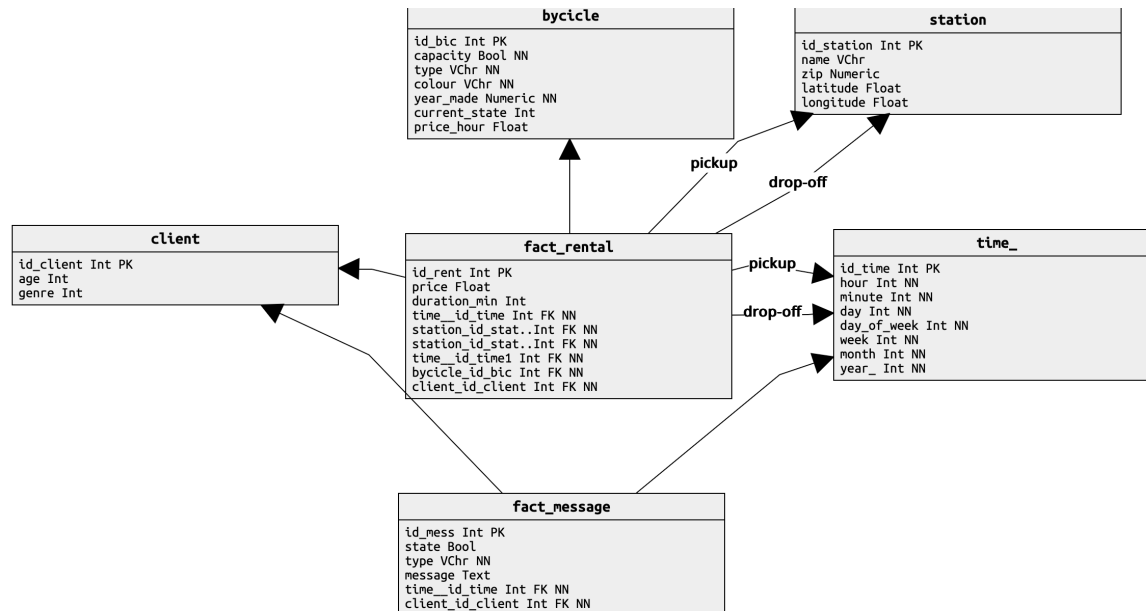
Finding the top-performing products and the states where the sales are coming from. In this case, you'll need to pull out each product name, order, and state and LIMIT to 10 as specified.

```
SELECT product_name, region_state,  
SUM(order_quantity) AS total_sales  
FROM sales s  
INNER JOIN products p ON s.product_id = p.id  
INNER JOIN stores st ON s.store_id = st.id  
INNER JOIN regions r ON st.state_code = r.state_code  
GROUP BY product_name, region_state  
ORDER BY total_sales DESC LIMIT 10;
```

Challenge 2 – index definition

What indexes should be used to optimize data access and why. Note that it is not enough to indicate the type of structures to use, it is necessary to concretize saying on which tables and fields these structures should be created.

1. Which are the busiest stations in terms of number of rentals and revenue? How does this evolve over the day/week/year?
2. Is there a significant difference between men and women clients, namely in terms of rental time, bike types and rental duration?



Challenge 2 – index definition

1. Which are the busiest stations in terms of number of rentals and revenue? How does this evolve over the day/week/year?

```
SELECT t.month, t.day_of_week, s.name "station",  
       count(*) as "number of rentals", sum(r.price) as "revenue"
```

```
FROM rentals r, time t, station s  
WHERE <JOIN_CLAUSES>  
AND t.year = 2024
```

```
GROUP BY s.name, t.month, t.day_of_week
```

```
ORDER BY 4,5;
```